CSAEC510 · PROMPT ENGINEERING · 5TH SEMESTER

Prompt Engineering
Unit II Study Guide

Standard Practices for Text Generation with ChatGPT, explained from zero, with diagrams, highlighted key points, exam tips and likely questions with answer plans.

Session 1 · Lists, Regex, JSON, YAML, Formats

Session 2 · Style, Chunking, Roles, Hallucinations

From messy text to data your code can use

```
Sure! Here are some
Disney characters:
1. Simba
2. Elsa
3. Bagheera (Jungle Book)
Hope this helps!
```

prompt engineering →

```
{
"characters": [
"Simba",
"Elsa",
"Bagheera"
]
}
```

Built from your Module 2 Session 1 & Session 2 class decks (98 slides)
and the textbook *Prompt Engineering for Generative AI*, James Phoenix & Mike Taylor (O'Reilly, 2024), Chapter 3

START HERE

0 How to Use This Guide

This guide covers **all of Unit II**, both sessions, every slide, rewritten in simple language with the textbook filling the gaps. It is the same Unit II as in the full CIE-1 guide, packaged on its own with a short Unit I primer so it stands alone. The coloured boxes tell you what each piece of text is for:

KEY IDEA
The core definition or idea. **Learn these word-for-word** — they make perfect opening lines.

EXAM TIP
How examiners ask about this topic, and how to score full marks.

REMEMBER
Short summaries and memory tricks to lock the idea in.

WATCH OUT
Common mistakes and traps. Read these twice.

EASY ANALOGY
Everyday comparisons that make hard ideas click.

BEYOND YOUR SLIDES
Extra material from the textbook that your slides only list, or that adds depth.

Prompt / Output boxes show real prompts: PROMPT is what you type, OUTPUT is what the AI replies. A red label means a bad prompt and a green label means a good one. Yellow highlights mark the phrases worth remembering.

Study plan — about 4 hours in 4 sittings

	Sitting	Topics	Time
☐	1	Primer + 2.1 – 2.5 · Lists, hierarchical lists, regex parsing (Examples 3-1, 3-2)	75 min
☐	2	2.6 – 2.12 · JSON, YAML, filtering, diverse formats, ELI5, translation, ask for context	60 min
☐	3	2.13 – 2.22 · Style unbundling, chunking, role prompting, hallucinations, thinking time	75 min
☐	4	R2 likely questions + cheat sheet + draw the diagrams from memory	30 min

FAST TRACK
Only have 1 hour? Read just the **Key idea**, **Remember** and **Exam tip** boxes, look at every diagram, then do R2 and the cheat sheet.

REVISION ROUTINE
Night before: cheat sheet + "Likely CIE questions" (R2). Draw from memory: regex pattern breakdown, YAML filter chart, ask-for-context loop, chunking table, with/without reference text.

Contents

UNIT II — STANDARD PRACTICES FOR TEXT GENERATION WITH CHATGPT

P Before You Start: 6 Ideas from Unit I 5
2.1 Generating Lists 5
2.2 Hierarchical List Generation 7
2.3 Parsing with Regular Expressions — Example 3-1 8
2.4 Keeping the Structure — Example 3-2 (Dictionary) 10
2.5 When to Avoid Using Regular Expressions 12
2.6 Generating JSON 12
2.7 YAML 14
2.8 Filtering YAML Payloads 15
2.9 Diverse Format Generation with ChatGPT 16
2.10 Explain It Like I'm Five (ELI5) 17
2.11 Universal Translation Through LLMs 17
2.12 Ask for Context 17
2.13 Text Style Unbundling 18
2.14 Identifying the Desired Textual Features 19
2.15 Generating New Content with the Extracted Features 20
2.16 Extracting Specific Textual Features with LLMs 20
2.17 Chunking Text and Strategies 20
2.18 Role Prompting 22
2.19 GPT Prompting Tactics 24
2.20 Avoiding Hallucinations with Reference 24
2.21 Give GPT "Thinking Time" and Inner Monologue 26
2.22 Self-Eval LLM Responses 26
R2 Unit II — Quick Revision & Likely Questions 28

FINAL REVISION

★ Unit II Cheat Sheet 31
A–Z Unit II Glossary 32
✓ Exam-Day Answer-Writing Tips 32

UNIT II

Standard Practices for Text Generation with ChatGPT

Practical techniques: getting clean lists, JSON and YAML that code can read, transforming and styling text, chunking long documents, role prompting and reducing hallucinations.

Generating Lists	2.1	Hierarchical List Generation	2.2
Parsing with Regular Expressions — Example 3-1	2.3	Keeping the Structure — Example 3-2 (Dictionary)	2.4
When to Avoid Using Regular Expressions	2.5	Generating JSON	2.6
YAML	2.7	Filtering YAML Payloads	2.8
Diverse Format Generation with ChatGPT	2.9	Explain It Like I'm Five (ELI5)	2.10
Universal Translation Through LLMs	2.11	Ask for Context	2.12
Text Style Unbundling	2.13	Identifying the Desired Textual Features	2.14
Generating New Content with the Extracted Features	2.15	Extracting Specific Textual Features with LLMs	2.16
Chunking Text and Strategies	2.17	Role Prompting	2.18
GPT Prompting Tactics	2.19	Avoiding Hallucinations with Reference	2.20
Give GPT "Thinking Time" and Inner Monologue	2.21	Self-Eval LLM Responses	2.22
Unit II — Quick Revision & Likely Questions	R2		

⏱ ~4 hours · 4 sittings

UNIT II · PRIMER

P Before You Start: 6 Ideas from Unit I

Unit II builds on a few Unit I ideas. If any of these is new to you, read this page first.

Token
The small piece of text an LLM reads and writes: a word, part of a word, or punctuation. LLMs generate answers **one token at a time**.

Context window
The maximum number of tokens a model can consider at once. This is why long documents need **chunking** (2.17).

Temperature
Controls randomness. Low = predictable and consistent (good for JSON/YAML). High = varied and creative.

Few-shot prompting
Putting a few **examples** of the desired output in the prompt. Unit II uses this constantly (example lists, example JSON).

Specify Format
One of the five principles: tell the AI **exactly how the answer should look** so code can read it.

Hallucination
Output that sounds right but is unsupported or false. Reference text reduces it (2.20).

REMEMBER

Unit II is mostly **Principle 2 (Specify Format)** and **Principle 3 (Provide Examples)** applied to real tasks, plus techniques to make answers more reliable.

UNIT II · SESSION 1 — STANDARD PRACTICES FOR TEXT GENERATION WITH CHATGPT

2.1 Generating Lists

KEY IDEA

Automatically generating lists is incredibly powerful: you focus on **higher-level tasks** while GPT can automatically generate, refine, rerank and deduplicate lists on your behalf.

The naive list prompt

PROMPT

```
Generate a list of Disney characters.
```

OUTPUT

```
Sure, here is a list of some popular Disney characters:

1. Mickey Mouse
2. Minnie Mouse
...
30. Bagheera (The Jungle Book)
```

This looks fine to a human, but it causes **5 problems for a computer program**:

#	Problem	What goes wrong
1	Different list format	GPT uses `1. Simba` but your Python expects `- Simba` → splitting on `-` fails.
2	Extra explanation	"Sure! Here are some Disney characters: ... Hope this helps!" — extra sentences make parsing harder.
3	Count not controlled	Asked for characters, GPT may give 10, 20, 30... If you need exactly 5, the number is **unpredictable**.
4	Inconsistent names	"Simba", "Elsa", "Bagheera (The Jungle Book)" — some have the movie in brackets → **inconsistent data** needing extra cleaning.
5	No filtering	"Only male characters" — GPT might still include characters that don't meet your exact rule.

The optimized list prompt

OPTIMIZED

```
Generate a bullet-point list of 5 male Disney characters.
Only include the name of the character for each line.
Never include the film for each Disney character.
Only return the Disney characters, never include any commentary.

Below is an example list:

* Aladdin
* Simba
* Beast
* Hercules
* Tarzan
```

OUTPUT

```
* Woody
* Buzz Lightyear
* Stitch
* Jack Sparrow
* Prince Charming
```

WHAT THE OPTIMIZED PROMPT ACHIEVED

- Restricted the list to a **fixed size of five**
- Generated **only male** characters
- Correctly **formatted** with bullet points
- **Removed** any preceding commentary

PROVIDE EXAMPLES

Simply rephrasing your prompt to include **examples (few-shot prompting)** can greatly impact the output.

Simple lists are fine for most tasks but are **less structured**. Three typical nested data structures: ① **nested text (hierarchical lists)** ② **JSON** ③ **YAML**.

2.2 Hierarchical List Generation

KEY IDEA

Hierarchical lists are useful when your desired output is nested — for example, a detailed article outline with headings and sub-headings.

PROMPT

```
Generate a hierarchical and incredibly detailed article outline on:

What are the benefits of data engineering.

See an example of the hierarchical structure below:

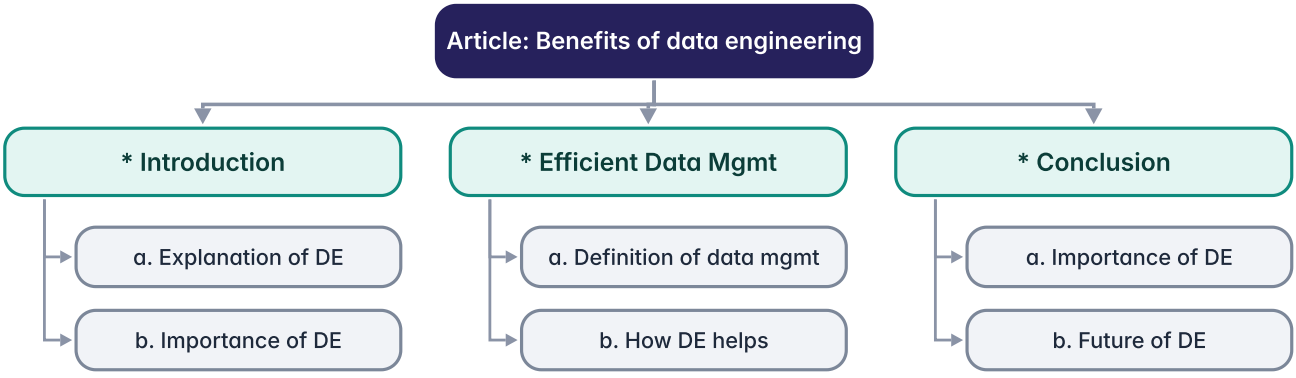
Article Title: What are the benefits of digital marketing?

* Introduction
    a. Explanation of digital marketing
    b. Importance of digital marketing in today's business world
* Increased Brand Awareness
    a. Definition of brand awareness
    b. How digital marketing helps in increasing brand awareness
```

OUTPUT

```
Article Title: What are the benefits of data engineering?

* Introduction
    a. Explanation of data engineering
    b. Importance of data engineering in today's data-driven world
...(10 sections later)...
* Conclusion
    a. Importance of data engineering in the modern business world
    b. Future of data engineering and its impact on the data ecosystem
```



Hierarchical = arranged in levels: main heading → sub-heading → details (DE = data engineering).

Why these prompt words matter

Word in the prompt	What it tells GPT
"hierarchical"	"Don't give me a flat list — give me **main headings and sub-headings**."
"incredibly detailed"	Produce a large, detailed answer (e.g. a. What is DE? b. History c. Role of data engineers...). Similar words: *very long, comprehensive, include at least 10 headings*.
the example structure	Shows the exact format: `*` for headings, `a.` `b.` for sub-headings → makes parsing possible.

WATCH OUT

Asking for 10 doesn't guarantee 10. GPT might return only 8 headings. So your Python program should **check the result** (e.g. `if number_of_headings == 10: print("Valid")`) or be designed to work with whatever number GPT returns.

Parsing — the key concept

DEFINITION

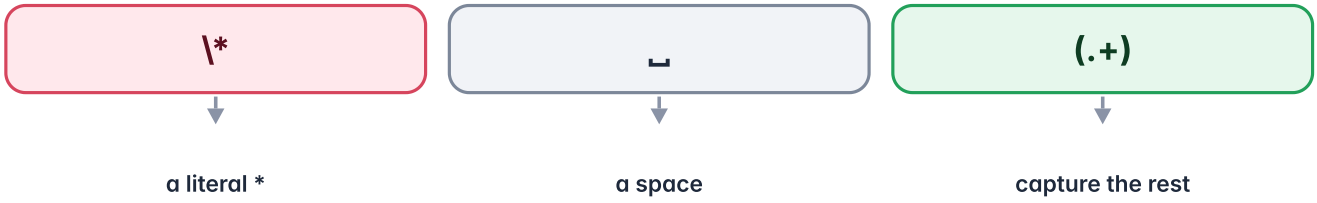
Parsing = converting unstructured text into structured data — taking GPT's text and turning it into a useful Python data structure.



2.3 Parsing with Regular Expressions — Example 3-1

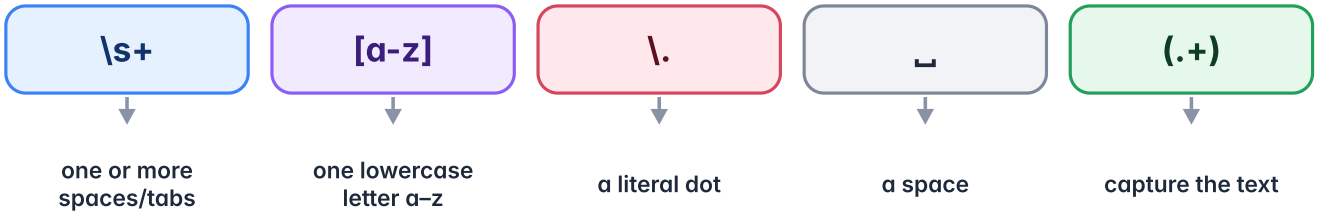
A **regular expression (regex)** is a pattern used to find specific text. Python's regex module is `re`. GPT's main headings start with `*` and sub-headings start with `a.`, `b.`, `c.` — so we can write one pattern for each. Imagine a very large outline: instead of extracting everything manually, regex finds all headings and sub-headings automatically.

Pattern 1 — headings: r'* (.+)'



" Introduction" → captures "Introduction"*

Pattern 2 — sub-headings: r'\s+[a-z]\. (.+)'



" a. Explanation of data engineering" → captures "Explanation of data engineering"

Symbol	Meaning	Why
`\*`	the literal * character	`*` has a special meaning in regex, so `\` "escapes" it: "I literally mean *"
`\.`	the literal . character	`.` normally means "any character"
`.`	any character	
`+`	one or more times	`.+` = one or more of any character
`( )`	capture group	the part we want to extract (read with `group(1)`)
`\s`	whitespace (space/tab)	`\s+` = one or more spaces · `\s*` = zero or more spaces
`[a-z]`	one lowercase letter	matches a, b, c ... z

EXAMPLE 3-1 · PARSING A HIERARCHICAL LIST

```
import re

openai_result = """
* Introduction
    a. Explanation of data engineering
    b. Importance of data engineering in today's data-driven world
* Efficient Data Management
    a. Definition of data management
    b. How data engineering helps in efficient data management
* Conclusion
    a. Importance of data engineering in the modern business world
    b. Future of data engineering and its impact on the data ecosystem
"""

# Regular expression patterns
heading_pattern = r'\* (.+)'
subheading_pattern = r'\s+[a-z]\. (.+)'

# Extract headings and subheadings
headings = re.findall(heading_pattern, openai_result)
subheadings = re.findall(subheading_pattern, openai_result)

# Print results
print("Headings:\n")
for heading in headings:
    print(f"* {heading}")

print("\nSubheadings:\n")
for subheading in subheadings:
    print(f"* {subheading}")
```

OUTPUT

```
Headings:
* Introduction
* Efficient Data Management
* Conclusion

Subheadings:
* Explanation of data engineering
* Importance of data engineering in today's
data-driven world
* Definition of data management
* How data engineering helps in efficient
data management
* Importance of data engineering in the
modern business world
* Future of data engineering and its impact
on the data ecosystem
```

`re.findall()` = find **ALL** matches, not just one. It returns a list:

```
["Introduction",
 "Efficient Data Management",
 "Conclusion"]
```

WATCH OUT

The problem: Example 3-1 produces **two separate lists**. It **doesn't know which sub-heading belongs to which heading** — the relationship is lost.

2.4 Keeping the Structure — Example 3-2 (Dictionary)

Example 3-2 solves the problem by building a **dictionary** where each heading maps to its own list of sub-headings:

GOAL

```
{
    "Introduction": [
        "a. Explanation of data engineering",
        "b. Importance of data engineering in today's data-driven world"
    ],
    "Efficient Data Management": [
        "a. Definition of data management",
        "b. How data engineering helps in efficient data management"
    ],
    "Conclusion": [ "a. Importance of ...", "b. Future of ..." ]
}
```

EXAMPLE 3-2 · PARSING INTO A DICTIONARY

```
import re

section_regex = re.compile(r"\* (.+)")              # headings: "* Introduction"
subsection_regex = re.compile(r"\s*([a-z]\..+)")    # sub-headings: "a. ..." (\s* = 0+ spaces)

result_dict = {}          # empty dictionary we will fill
current_section = None    # remembers which heading we are under

for line in openai_result.split("\n"):         # go through GPT's text line by line
    section_match = section_regex.match(line)
    subsection_match = subsection_regex.match(line)

    if section_match:                                  # is this line a heading?
        current_section = section_match.group(1)       # "Introduction"
        result_dict[current_section] = []              # {"Introduction": []}
    elif subsection_match and current_section is not None:   # a sub-heading under a heading?
        result_dict[current_section].append(subsection_match.group(1))

print(result_dict)
```

Line-by-line walkthrough

Line read	Matches?	Action	result_dict after
`* Introduction`	heading	current_section = "Introduction"; create empty list	`{"Introduction": []}`
`a. Explanation…`	sub-heading	append to current section	`{"Introduction": ["a. Explanation…"]}`
`b. Importance…`	sub-heading	append to current section	`{"Introduction": ["a. …", "b. …"]}`
`* Efficient Data Mgmt`	heading	current_section changes; new empty list	`{…, "Efficient Data Management": []}`
`a. Definition…`	sub-heading	append under the **new** heading	`{…, "Efficient…": ["a. Definition…"]}`

KEY POINTS

- `re.compile()` prepares a pattern so it can be used **repeatedly**.
- `\s*` (zero or more spaces) handles both `" a. Explanation"` and `"a. Explanation"` → **more flexible** than `\s+`.
- `group(1)` returns the text captured by the first `( )`.
- `current_section is not None` ensures we never add a sub-heading before any heading has been seen.

2.5 When to Avoid Using Regular Expressions

KEY IDEA

Regex is great for **simple patterns**, but as the data becomes more complicated, regex becomes hard to write and maintain. Instead, ask GPT to return a **structured format** like JSON, YAML, CSV, Mermaid or code.

	Regex approach	JSON approach
What you do	Write patterns like `r'\* (.+)'` and `r'\s+[a-z]\. (.+)'`	GPT already returns `{"Introduction": […], "Conclusion": […]}`
Parsing	Custom code; breaks if the format changes slightly	One line: `json.loads()`
Complex data	Patterns become complicated too	Usually **easier and cleaner**
Best for	Simple patterns	Structured / nested data

2.6 Generating JSON

Tell GPT: *"Give me the answer **only in valid JSON**."* You can also give an **example of the expected JSON** — that is **giving direction and examples**, and GPT follows a format you show it more reliably.

PROMPT

```
Compose a very detailed article outline on "The benefits of learning code" with a JSON payload structure that highlights key points.

Only return valid JSON.

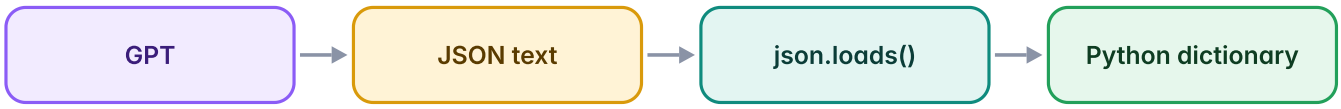
Here is an example of the JSON structure:
{
  "Introduction": [
    "a. Explanation of data engineering",
    "b. Importance of data engineering in today's data-driven world"],
  ...
  "Conclusion": [
    "a. Importance of data engineering in the modern business world",
    "b. Future of data engineering and its impact on the data ecosystem"]
}
```

OUTPUT (VALID JSON)

```
{
  "Introduction": [
    "a. What is coding?",
    "b. Importance of coding"
  ],
  "Conclusion": [
    "a. Benefits of coding",
    "b. Future of coding"
  ]
}
```

Why is JSON useful?

Plain text ("Introduction: Coding is important... Conclusion: Coding has many benefits...") makes it hard for Python to know where each part begins and ends. JSON has a **fixed structure**, so Python can directly access `data["Introduction"]`.



JSON.LOADS() — THE EASY PART

```python
import json

openai_json_result = """
{
    "Introduction": [
        "What is coding?",
        "Importance of coding"
    ]
}
"""
parsed_json_payload = json.loads(openai_json_result)   # JSON string -> Python dict
print(parsed_json_payload["Introduction"])
```

Common JSON problems

PROBLEM 1 — EXTRA TEXT

GPT might wrap the JSON in chatty text and code fences:

`Sure! Here's the JSON:`

`` ```json {"Name": "John Smith"} ``` ``

A human understands it, but `json.loads()` expects **only** JSON. Ideally GPT returns just `{"Name": "John Smith"}`.

PROBLEM 2 — INVALID JSON

GPT might produce **invalid JSON**, e.g. `{"Name": "John Smith", "Age":}`. `json.loads()` will raise an error. Always assume the output might be invalid and handle it.

GOOD PRACTICE (SAFE PARSING)

```
try:
    data = json.loads(gpt_output)
except json.JSONDecodeError:
    data = None      # e.g. retry the request or ask GPT to fix its JSON
```

2.7 YAML

KEY IDEA

YAML (also .yml) is another way of storing structured information — the same kind of data as JSON but in a simpler-looking, **indentation-based** format that is easy for humans to read.

JSON

```
{
  "item": "Apple",
  "quantity": 5,
  "unit": "pieces"
}
```

YAML (SAME DATA)

```
# a shopping item
item: Apple
quantity: 5
unit: pieces
```

A YAML LIST OF ITEMS

```
- item: Apple Slices
  quantity: 5
  unit: pieces

- item: Eggs
  quantity: 2
  unit: dozen
```

① **No lots of special characters**

JSON uses `{ } [ ] , "`; YAML mostly uses **indentation** → easier to write.

② **Readability**

Designed to be easy for humans: `item: Milk` / `quantity: 1` / `unit: gallon`.

③ **Comments**

YAML allows comments with `#`, e.g. `# This is the quantity of milk` — useful for teams.

EXAM TIP

"Compare JSON and YAML" → write the same data in both, then list YAML's 3 advantages (fewer special characters, readability, comments) and JSON's strength (strict structure that every programming language parses easily).

2.8 Filtering YAML Payloads

KEY IDEA

GPT can act as a data cleaner / filter: give it a **schema** (the allowed items) and the user's request, and it keeps only what matches.

PROMPT*

```
You are a helpful assistant that converts a user's shopping list into YAML.
Only use items from the schema below. Remove anything that is not in the schema.
If nothing matches, return "No Items".

Schema (allowed items): Apple Slices, Milk, Bread, Eggs

User query: "5 apple slices, 2 dozen eggs and 3 bananas"
```

OUTPUT

```
- item: Apple Slices
  quantity: 5
  unit: pieces

- item: Eggs
  quantity: 2
  unit: dozen
```

*Simplified version of the textbook's prompt. **Bananas** isn't in the allowed list, so GPT filters it out. If the user only says "5 bananas", GPT returns **"No Items"**.

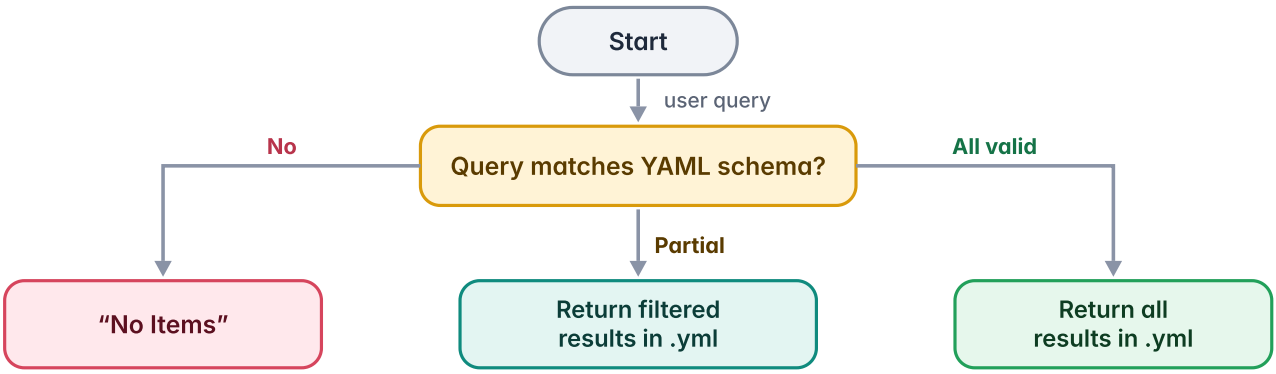


Figure 3-1: using an LLM to decide the control flow of an application instead of code

LLM as a reasoning engine

NORMALLY: PROGRAMMING LOGIC

```
if item in allowed_items:
    keep_item()
else:
    remove_item()
```

Instead, you can give GPT **instructions** and let it perform this filtering. So an LLM can sometimes perform tasks that would normally require programming logic — even deciding the **control flow** of an app.

WATCH OUT

For important applications, you should **still validate the LLM's output with code**.

2.9 Diverse Format Generation with ChatGPT

GPT isn't limited to JSON and YAML. It can generate **JSON · YAML · CSV · Mermaid diagrams · Python code · JavaScript code · conversations · scripts** and more.

Mermaid diagrams

MERMAID CODE GENERATED BY GPT

```
graph TD
ChooseFood[Choose Food] --> AddToCart[Add to Cart]
AddToCart --> ConfirmCart[Confirm Cart]
ConfirmCart --> PayForMeal[Pay for Meal]
```

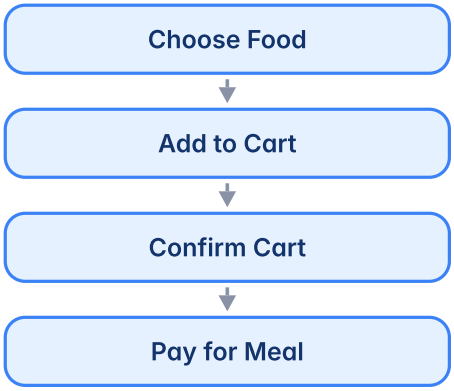


Figure 3-2: the rendered flowchart

Mermaid is a language used to create diagrams from text. A Mermaid renderer turns the code into a flowchart.

Normal language → GPT → Mermaid code → Diagram

Generating CSV and mock CSV data

CSV OUTPUT

```
name,age,grade
John,16,A
Alex,17,B
Emma,16,A
```

CSV = Comma-Separated Values. The comma separates the columns (name , age , grade → column column column). Say "Generate CSV" and GPT knows to use commas.

Mock CSV data = realistic but fake data generated for **testing** an app or demo — e.g. "Generate 5 rows of mock customer data as CSV with columns name, age, city." No real (private) data needed.

IMPORTANT LESSON ABOUT FORMATS

GPT **heavily depends on the format you specify**: "Give me CSV" → commas; "Give me JSON" → JSON syntax; "Give me Mermaid" → Mermaid syntax. Be very clear about the desired format.

Diffusion models — other modalities

A **diffusion model** is a type of generative AI commonly used to **create images** ("Create a picture of a cat wearing sunglasses" → diffusion model → image). When language models and image models work together we can move between forms of information — **Text → Image, Text → Code, Text → Diagram, Text → Story**. This is working with different **modalities/formats**.

2.10 Explain It Like I'm Five (ELI5)

KEY IDEA

ELI5 is a prompting technique: tell GPT *"Explain this like I'm 5 years old."* It means use very simple words and examples so even a beginner can understand.

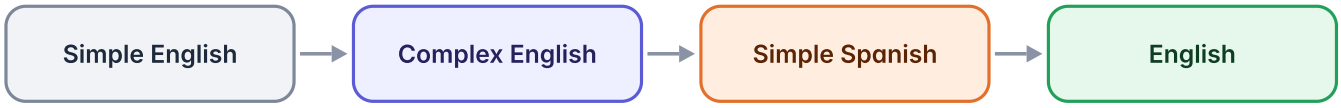
DIFFICULT
```
Cancer involves abnormal
proliferation of neoplastic
cells...
```

ELI5
```
Cancer is a sickness where some
cells in the body grow in a way
they shouldn't.
```

ELI5 is useful for: **technical documents · research papers · scientific concepts · difficult textbooks · complex programming topics**.

2.11 Universal Translation Through LLMs

LLMs can translate between languages — and between **levels of complexity**. The textbook example chains several transformations:



ENGLISH `The sun rises and birds sing.`

SPANISH `El sol sale y los pájaros cantan.`

SPANISH `El sol sale y los pájaros cantan.`

BACK `The sun rises and the birds sing.`

But translation isn't perfect

- **Meaning can be lost**: Original → Translation → Another transformation → Translation back — each step can slightly change the meaning.
- **High- vs low-resource languages**: LLMs perform better in languages with lots of training data; languages with fewer resources may give poorer translations.
- **Same for programming languages**: better at popular ones (Python, JavaScript, Java, C++); less reliable with very new languages, rare languages and new libraries/packages — because the model has seen more examples of popular technologies.

2.12 Ask for Context

KEY IDEA

When GPT doesn't have enough information, let it ask you for the context it needs before answering — instead of guessing.

BAD

```
Should I use MongoDB or PostgreSQL?
```

OUTPUT

```
"It depends on your requirements."
← not very useful
```

BETTER

```
Should I use MongoDB or PostgreSQL?

First, tell me what information you need to make a good recommendation.
```

GPT might ask you to consider: what type of data · is it structured · how much data · complex relationships? · how many users · performance needed · budget · high availability · security requirements · preferred technology. Then you provide it:

YOU

```
My application has:
- Structured customer data
- Many relationships between tables
- Complex queries
- 100,000 users
```

RESULT

```
Now GPT can give a much more informed recommendation (here: PostgreSQL).
```

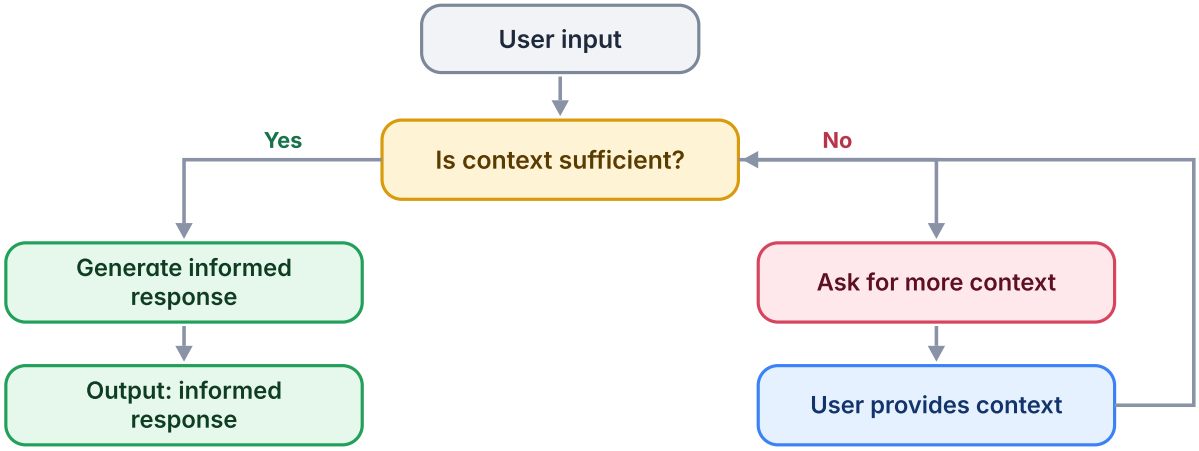


Figure 3-3: the decision process of an LLM while asking for context

EXAM TIP

Draw this loop diagram in the exam — "Is context sufficient? Yes → informed response; No → ask for more context → user provides context → check again."

2.13 Text Style Unbundling

DEFINITION

Text style unbundling = identifying the writing characteristics of a text (tone, length, vocabulary, structure, content) so that **new content can be created in a similar style**.

We look at a piece of writing and find out **how** it is written, separating its style into parts:

TEXT

```
AI is changing our world. It
creates exciting opportunities,
but we must use it responsibly.
```

Feature	Analysis
Tone	Positive + informative
Length	Short
Vocabulary	Simple
Structure	Intro → benefit → warning

Then we ask GPT to write another article in the **same style**.

Why is it useful?

Imagine a company with **100 writers** who all need to write in the same style. Style unbundling gives them a **style guide** so that everyone writes consistently.

1
Improved brand consistency
All company content sounds similar.

2
Faster content creation
Writers know exactly what style to follow.

3
Adaptability
Same style for blogs, emails, ads, social media, etc.

2.14 Identifying the Desired Textual Features

Before copying a writing style, decide **which parts of the style** you want to copy:

Feature	Meaning	Examples
1. Tone of voice	The feeling / attitude of the writing	Formal "We would like to inform you..." · Casual "Hey! Here's what you need to know." · Funny "Well, that didn't go as planned!" · Professional "The results demonstrate significant improvement."
2. Length	How long the new content should be	Short 100 words · Medium 500 words · Long 2000 words
3. Vocabulary & phrasing	What kind of words the writer uses	Simple "The system works quickly." · Technical "The system demonstrates high computational efficiency."
4. Structure	How the content is organised	Title → Introduction → Main points → Examples → Conclusion
5. Content	Which ideas the text usually focuses on	An AI article: explain AI → benefits → risks → ethical use

EXAM TIP

Memory hook for the 5 features: **"T-L-V-S-C"** → **T**one, **L**ength, **V**ocabulary, **S**tructure, **C**ontent ("**T**all **L**adies **V**isit **S**mall **C**afés").

2.15 Generating New Content with the Extracted Features

After identifying the style, use it to create new content. Suppose GPT's analysis is: Tone → informative and optimistic; Length → short; Vocabulary → simple; Structure → Problem → Benefits → Concerns → Conclusion. Now:

PROMPT

```
Write a new article about cybersecurity using the same tone, length, vocabulary, and structure.
```

OUTPUT

```
A short, optimistic, simply-worded article on cybersecurity that follows Problem → Benefits → Concerns → Conclusion.
```



Basic process of text style unbundling

2.16 Extracting Specific Textual Features with LLMs

We don't always need the whole style — we can ask GPT to extract specific information with a clear instruction.

PROMPT

```
Extract the entities from: "Artificial intelligence is transforming healthcare and entertainment."
```

OUTPUT

```
Entities:
- Artificial Intelligence
- Healthcare
- Entertainment
```

LLMs can identify: **names · organizations · locations · products · dates · topics · sentiment · tone · keywords**.

2.17 Chunking Text and Strategies

KEY IDEA

Chunking = breaking a large piece of text into smaller pieces (chunks). Book → chapters → paragraphs → sentences.

Why? LLMs have limits on how much text they can process at once (the **context length**). A huge document = too many tokens → the LLM may reject or cut off the request. Chunking solves this.

✓ WHEN TO USE CHUNKING

- **Large documents** — too big for the model (e.g. a 500-page book)
- **Complex analysis** — deep analysis of a large document
- **Multiple-topic documents** — AI / cybersecurity / cloud in one file → separate chunks

× WHEN NOT TO USE CHUNKING

- **Short documents** — already small
- **Simple analysis** — don't make a simple task complicated
- **Single-topic documents** — small & focused, little benefit

× Poor chunking — split by word

The | local | council | increased | the | education | budget

meaning & relationships between words are lost
many tiny pieces → more processing

✓ Better chunking — split by sentence

The local council increased the education budget by 10%.

The additional funds will improve school infrastructure.

Some critics argue that the increase isn't enough.

Poor chunking causes loss of context and increased processing. Each good chunk contains meaningful information.

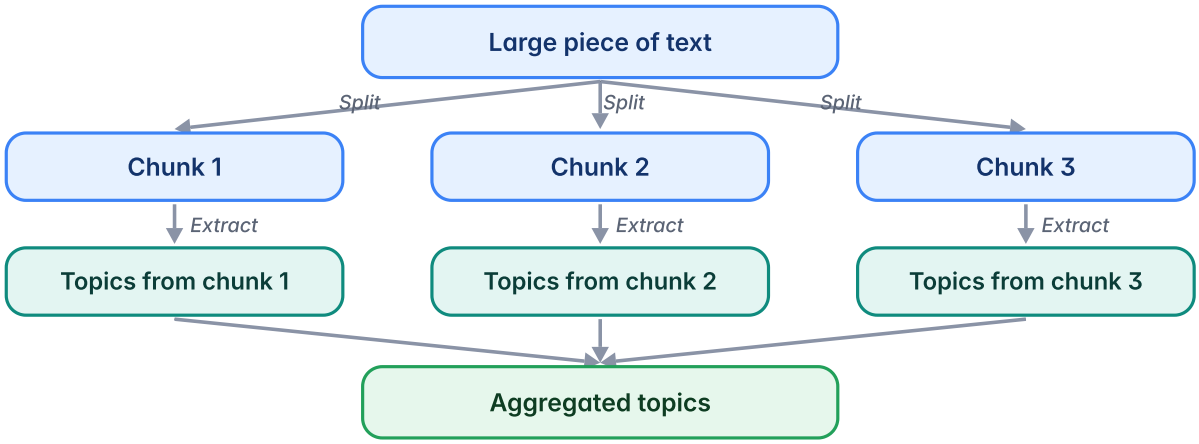


Figure 3-5: topic extraction with an LLM after chunking text

The six chunking strategies

Strategy	How	Advantages	Disadvantages	Useful for
① By sentence	Each sentence = a chunk	Preserves context; suitable for various tasks	May not be efficient for very long content	Summarization, translation, sentiment analysis
② By paragraph	Each paragraph = a chunk	Handles longer content; keeps related ideas together (cohesive units)	Less granularity; may miss subtle connections	Document analysis, topic analysis, information extraction
③ By topic / section	Group text by topic (AI → chunk	Identifies main themes; better for classification	Requires topic identification; may miss fine details	Classification, recommendations, clustering

Strategy	How	Advantages	Disadvantages	Useful for
	1, cyber → chunk 2)			
④ **By complexity**	Easy / medium / technical content	Groups similar complexity levels; adaptive	Requires complexity measurement; not for all tasks	Readability analysis, personalised learning, content adaptation
⑤ **By length**	Fixed size, e.g. 1000 words each	Manages very long content; efficient processing	Loss of context — a sentence/idea may be cut in the middle	Very large documents
⑥ **By tokens**	A tokenizer (e.g. **tiktoken**) splits by token count	Accurate token counts → avoids hitting prompt token limits	Requires tokenization; adds computational complexity	Staying within the model's token limit

COUNTING TOKENS WITH TIKTOKEN (OPENAI'S TOKENIZER)

```
import tiktoken

encoding = tiktoken.get_encoding("cl100k_base")   # tokenizer used by GPT-3.5 / GPT-4
tokens = encoding.encode("Chunking text is important.")
print(len(tokens))                                # how many tokens this text uses
```

EXAM TIP

Table 3-1 (six strategies with advantages & disadvantages) is a favourite 10-mark question. Memory hook: **"S-P-T-C-L-T"** → **S**entence, **P**aragraph, **T**opic, **C**omplexity, **L**ength, **T**okens.

2.18 Role Prompting

KEY IDEA

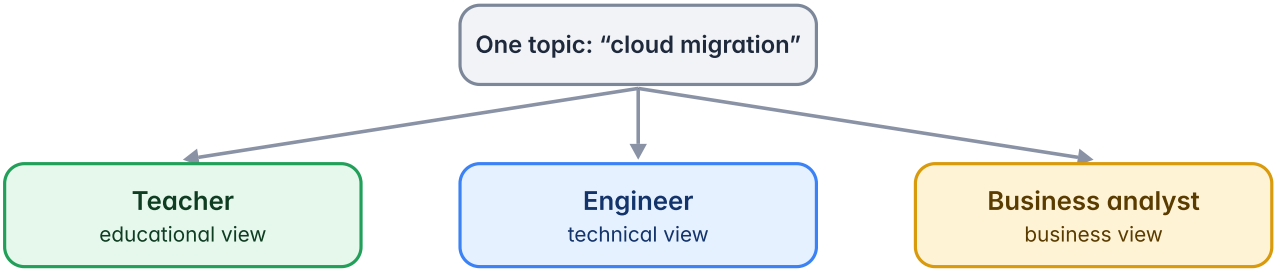
Role prompting = telling the AI to "act as" a particular type of person or expert — e.g. "Act as a teacher", "Act as a Python expert", "Act as a technical reviewer".

PLAIN `Review MongoDB.`

ROLE
```
Act as a technology reviewer. Give
an in-depth review of MongoDB,
including features, advantages,
disadvantages, and comparisons.
```

Benefits of role prompting (6)

Benefit	Explanation / example
① **More focused answers**	The AI has a specific role: general GPT → broad response; tech reviewer → technical review.
② **Specific expertise**	"Act as a programming instructor" / "Act as a business analyst" — the role sets the context.
③ **Control response style**	Formal, casual, humorous, professional, educational — "Act as a friendly teacher and explain recursion."
④ **Encourage creativity**	"Act as a storyteller and explain the history of computers."
⑤ **Explore different perspectives**	Teacher → educational, engineer → technical, business analyst → business view.
⑥ **Improve user engagement**	Feels interactive — "Act as an interviewer and ask me Python interview questions."



Benefit ⑤: the same topic seen from different roles

Challenges of role prompting (3)

① Bias and stereotypes
Some roles carry stereotypes; assigning a role based on a stereotype can cause inappropriate or biased responses.

② Role drift
In a long conversation the AI may slowly stop following the role ("Act as a Python teacher" ... later talks about something unrelated).

③ Incorrect information
Calling the AI an "expert" doesn't make it correct. Role prompting changes the **style/context**, not the **reliability** of the facts.

When to use role prompting (5)

Use it when you need...	Example prompt	Good for
① **Specific expertise**	Act as a database expert.	Technical topics
② **A specific response style**	Act as a friendly teacher.	Learning
③ **Creative responses**	Act as a storyteller.	Stories, creative writing
④ **Different perspectives**	Analyze this problem from a software engineer's perspective.	Examining aspects of a topic
⑤ **More engaging conversations**	Act as an interviewer and conduct a mock interview.	Practice, interaction

2.19 GPT Prompting Tactics

This section collects techniques for getting better results. The goal: give the model better instructions and better information so the output is more useful and reliable.

Role prompting
"Act as ..." (2.18)

Text style unbundling
copy a writing style (2.13)

Asking for context
let GPT ask questions (2.12)

Reference text
answer from a given document (2.20)

Thinking time
solve → check → answer (2.21)

Task decomposition
break big tasks into steps (1.10)

2.20 Avoiding Hallucinations with Reference

KEY IDEA

A **hallucination** is when the model gives information that sounds correct but is actually unsupported or incorrect. Supplying **reference text** and telling GPT to answer **only** from it reduces hallucinations.

You give GPT an article that only says *"B2B customers usually have longer sales cycles."* and ask *"Are B2C sales more cost-effective?"* The article doesn't say that. Instead of guessing, instruct GPT:

PROMPT

```
Only answer using the provided article. If the answer isn't there, say "I could not find an answer."

Article: """B2B customers usually have longer sales cycles."""

Question: Are B2C sales more cost-effective?
```

OUTPUT

```
I could not find an answer.
```

How reference text works

PROMPT

```
Reference: A nuclear reactor is a device that initiates, controls and maintains a nuclear chain reaction.

Question: What is a nuclear reactor?
```

OUTPUT

```
A nuclear reactor is a device that controls and maintains a nuclear chain reaction.
```

Adding citations

PROMPT

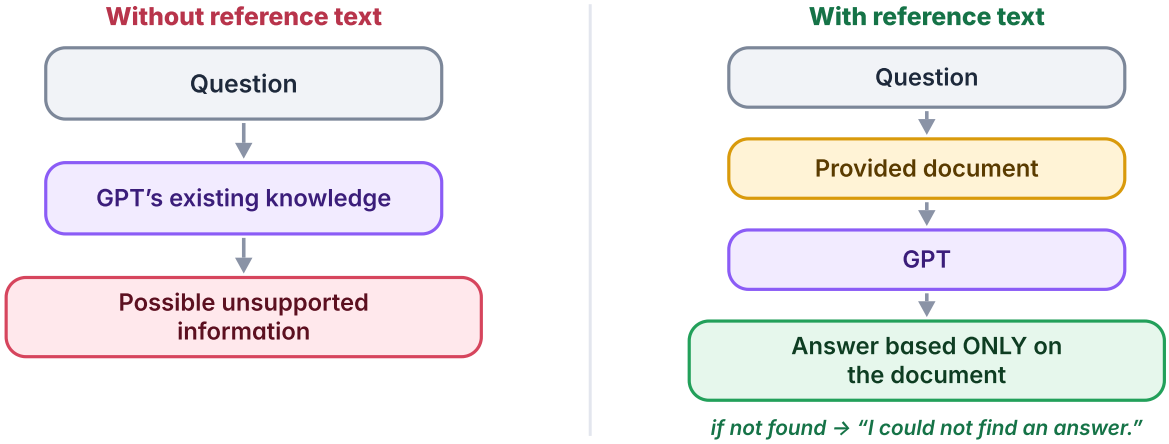
```
Answer only using the provided document and include the passage used as evidence.
```

OUTPUT

```
Answer: A nuclear reactor is a device that controls and maintains a nuclear chain reaction.

Citation: "A nuclear reactor is a device that initiates, controls, and maintains a nuclear chain reaction."
```

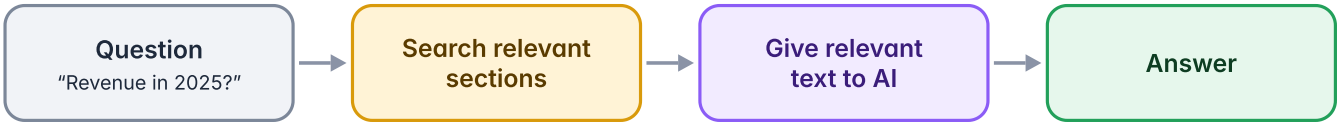
Citations make it easy to **check where the answer came from**.



Why reference text is useful: it gives you control over the source of the answer.

Dynamic retrieval of relevant information

GPT has a **limited context window**, so it can't always read an entire huge document. Instead: **store** the large document → **search** for parts relevant to the question → give **only those parts** to the AI → ask it to answer using them.



Example: a 500-page company report — send only the revenue section, not all 500 pages. This is Retrieval-Augmented Generation (RAG), covered fully in Unit IV.

2.21 Give GPT “Thinking Time” and Inner Monologue

KEY IDEA

Tell the model to work through the problem carefully before giving the final answer — e.g. “First solve the problem yourself. Then compare your answer with the proposed solution.” The idea: **Solve → Check → Answer**.

PROMPT

```
Jane has 5 apples. John takes 2 apples from Mary. John gives those 2 apples to Jane.
How many apples does Jane have?

First work out your own solution step by step, then answer.
```

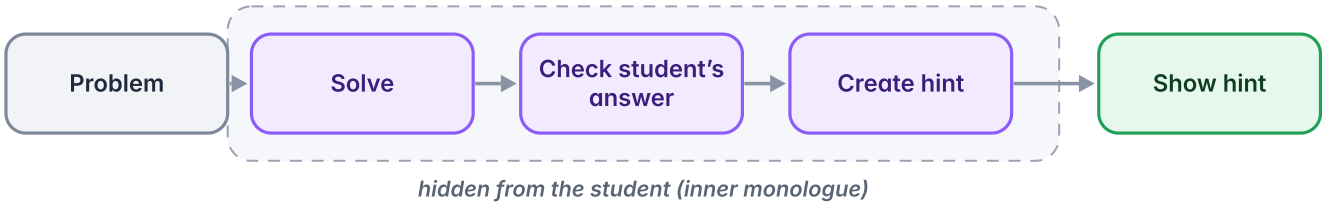
OUTPUT

```
Jane starts with 5.
John gets 2 from Mary.
John gives those 2 to Jane.
Jane now has 5 + 2 = 7.
```

The prompt makes the model check the problem carefully instead of immediately accepting a proposed answer.

Inner monologue tactic

Let the model do **intermediate work**, but don’t show those intermediate steps to the user. In a tutoring app, the AI internally solves the problem, checks the student’s answer, finds the mistake and creates a hint — the student sees **only the hint**.



Problem → Solve → Check student’s answer → Create hint → Show hint

PROMPT

```
Student answer: Probability = 13/204  (drawing a Heart and a Club)
```

SHOWN

```
Hint: Remember that the two cards can be drawn in either order. Consider both Heart
→ Club and Club → Heart.
```

Why hide the reasoning? It gives guidance **without revealing the complete solution** — the student thinks and learns (e.g. “Hint: start by calculating the probability of getting a heart on the first draw.”).

2.22 Self-Eval LLM Responses

KEY IDEA

Self-evaluation = asking the AI to review and improve its own previous answer: Generate → Review → Improve.

1ST PROMPT: “GENERATE A PYTHON HELLO WORLD FUNCTION.”

```python
def print_hello_world():
    print("Hello, World!")
```

2ND PROMPT: “CAN YOU IMPROVE IT? ADD TYPE HINTS OR DEFAULTS.”

```python
def print_message(message: str = "Hello, World!") -> None:
    """Print the given message."""
    print(message)
```

Improvement	Meaning
Type hint `message: str`	`message` should normally be a string
Return type `-> None`	the function does not return a value
Default value `= "Hello, World!"`	`print_message()` prints “Hello, World!”; `print_message("Good morning!")` prints “Good morning!” → more flexible
Docstring `"""…"""`	documents what the function does

R2 Unit II — Quick Revision & Likely Questions

SESSION 1 IN 6 LINES

- Lists: control **count, filter, format, no commentary** + give an example list.
- Hierarchical list → parse with regex: `r'\*(.+)'`, `r'\s+[a-z]\. (.+)'`.
- Example 3-1 = two separate lists; Example 3-2 = dictionary (keeps structure).
- Avoid regex for complex data → ask for JSON / YAML.
- JSON: "only valid JSON" + example; `json.loads()`; handle preamble/invalid JSON.
- YAML: readable, fewer symbols, comments; filter against a schema → "No Items".

SESSION 2 IN 7 LINES

- ELI5 = explain simply.
- Translation chains lose meaning; low-resource languages weaker.
- Ask for context: "What info do you need?"
- Style unbundling: Tone, Length, Vocabulary, Structure, Content.
- Chunking: 6 strategies — sentence, paragraph, topic, complexity, length, tokens.
- Role prompting: 6 benefits, 3 challenges, 5 uses.
- Reference text + "I could not find an answer" → fewer hallucinations; add citations.

REGEX IN 4 LINES

- `\*` and `\.` = literal * and .
- `(.+)` = capture one-or-more characters.
- `\s*` vs `\s+` = zero-or-more vs one-or-more spaces.
- `findall` = all matches; `group(1)` = captured text.

COMMON MISTAKES

- Asking for 10 items ≠ getting 10 — validate.
- GPT output may be invalid JSON — use try/except.
- Word-level chunking destroys context.
- Role prompting doesn't make facts correct; watch for role drift.

Likely CIE questions — with answer plans

Explain hierarchical list generation and how the output is parsed using regular expressions (Example 3-1 / 3-2). — 10 marks

ANSWER PLAN

- Prompt with "hierarchical", "incredibly detailed" + example structure; output outline; tree diagram.
- Parsing definition.
- Explain both patterns token-by-token; `findall`; output.
- Limitation of 3-1 → dictionary approach in 3-2 with walkthrough table.

When should you avoid regular expressions? Explain generating JSON and the common problems. 8 marks

ANSWER PLAN

- Regex hard to maintain for complex data → structured formats.
- Prompt: only valid JSON + example; json.loads flow.
- Problems: preamble/code fences, invalid JSON → try/except.
- Regex vs JSON table.

What is YAML? Compare with JSON. Explain filtering YAML payloads. 8 marks

ANSWER PLAN

- Same data in JSON and YAML; 3 advantages.
- Schema of allowed items; user query; filtered YAML; "No Items".
- Figure 3-1 decision diagram; LLM as reasoning engine; still validate with code.

Write notes on diverse format generation: Mermaid, CSV, mock CSV data, ELI5, universal translation. 8 marks

ANSWER PLAN

- Mermaid code → flowchart.
- CSV columns; mock data for testing.
- ELI5 example (cancer).
- Translation chain + limitations.

What is "Ask for Context"? Explain with an example and the decision diagram. 5 marks

ANSWER PLAN

- MongoDB vs PostgreSQL bad/better prompts; list of questions GPT asks; Figure 3-3 loop.

Explain text style unbundling: identifying textual features and generating new content. 8 marks

ANSWER PLAN

- Definition + example analysis.
- Benefits: consistency, speed, adaptability.
- 5 features with examples.
- Process pipeline; cybersecurity example; extracting specific features (entities).

What is chunking? Explain the six chunking strategies with advantages and disadvantages. 10 marks

ANSWER PLAN

- Definition + why (context length).
- When to use / not use.
- Poor vs good chunking example.
- Table 3-1 (6 strategies) + tiktoken; topic extraction diagram.

Explain role prompting — benefits, challenges and when to use it. 8 marks

ANSWER PLAN

- Definition + MongoDB review example.
- 6 benefits, 3 challenges (bias, role drift, incorrect info), 5 use cases.

How can hallucinations be avoided? Explain reference text, citations, dynamic retrieval, thinking time and self-evaluation. 10 marks

ANSWER PLAN

- Hallucination definition; B2B example with “I could not find an answer”.
- Nuclear reactor + citations; with/without diagram.
- Dynamic retrieval (RAG preview).
- Thinking time (Jane’s apples), inner monologue (tutoring hint).
- Self-eval (Hello World with type hints/defaults).

LAST-MINUTE REVISION

★ Unit II Cheat Sheet

CLEAN LISTS

Fixed count · filter (e.g. male only) · bullet format · no commentary · give an example list

HIERARCHICAL LISTS

"hierarchical" + "incredibly detailed" + example structure · asking for 10 ≠ getting 10 → validate

REGEX

`\* (.+)` headings · `\s+[a-z]\. (.+)` sub-headings · `findall` = all matches · `group(1)` = captured text · 3-1 = two lists, 3-2 = dict

AVOID REGEX WHEN…

Regex for simple patterns only → complex data: ask for JSON / YAML

JSON

"Only return valid JSON" + example · `json.loads` → dict · problems: chatty preamble, code fences, invalid JSON → try/except

YAML

Same data as JSON · fewer symbols · readable · comments (#) · filter against schema → "No Items" / filtered / all

DIVERSE FORMATS

JSON · YAML · CSV · Mermaid · Python/JS code · mock CSV for testing · diffusion models = images

ELI5 & TRANSLATION

ELI5 = simple words · translation chains lose meaning · low-resource languages weaker

ASK FOR CONTEXT

"What information do you need first?" · context sufficient? yes → answer; no → ask → user provides

STYLE UNBUNDLING

Tone · Length · Vocabulary · Structure · Content (T-L-V-S-C) · benefits: consistency, speed, adaptability

6 CHUNKING STRATEGIES

Sentence · Paragraph · Topic · Complexity · Length · Tokens (tiktoken) · never split by word

ROLE PROMPTING

6 benefits: focus, expertise, style, creativity, perspectives, engagement · 3 challenges: bias, role drift, wrong info

ANTI-HALLUCINATION

Answer ONLY from reference · "I could not find an answer" · cite the passage · retrieve relevant chunks (RAG)

MORE TACTICS

Thinking time (solve → check → answer) · inner monologue (hide steps, show hint) · self-eval (generate → review → improve)

LOOK UP ANY TERM

A–Z Unit II Glossary

Chunking — splitting large text into smaller meaningful pieces.

Citation — the passage from the reference text that supports an answer.

Context window — maximum tokens a model can consider at once.

CSV — Comma-Separated Values: rows of data with commas between columns.

Diffusion model — generative model commonly used to create images.

Dynamic retrieval — searching a large document and passing only relevant parts to the AI.

ELI5 — "Explain it like I'm five": the simplest possible explanation.

Entity extraction — pulling names, places, organisations, topics etc. out of text.

Few-shot — prompt containing several examples.

findall() — regex function returning every match as a list.

group(1) — the text captured by the first ( ) in a regex.

Hallucination — confident but unsupported or false output.

Hierarchical list — nested list of headings and sub-headings.

Inner monologue — model reasons internally; only the final part is shown.

JSON — structured key-value data format; parsed with json.loads().

json.loads() — converts a JSON string into a Python dictionary/list.

Mermaid — text language that renders into diagrams/flowcharts.

Mock data — realistic fake data generated for testing.

Modality — a type of information: text, image, audio, code, diagram.

Parsing — converting unstructured text into structured data.

RAG — Retrieval-Augmented Generation: retrieve relevant text, then answer.

re.compile() — prepares a regex pattern for repeated use.

Reference text — a document the model must answer from.

Regex — pattern for finding specific text (Python module re).

Role drift — AI gradually stops following its assigned role.

Role prompting — "Act as ..." instruction to set expertise/style.

Schema — definition of allowed structure/items (e.g. YAML filter).

Self-evaluation — asking the AI to review and improve its own answer.

Style unbundling — extracting tone, length, vocabulary, structure, content of a text.

Thinking time — asking the model to work the problem out before answering.

tiktoken — OpenAI's tokenizer library for counting tokens.

Type hint — annotation like message: str or → None in Python.

YAML — indentation-based, human-readable data format with comments.

SCORE MORE WITH THE SAME KNOWLEDGE

Exam-Day Answer-Writing Tips

Start with a definition, end with why

Open with the **Key idea** wording; close with one line on reliability, consistency or fewer hallucinations.

Always draw a diagram

Regex breakdown, YAML filter chart, ask-for-context loop, chunking flow, with/without reference text.

Show prompts and code

A bad vs good prompt in two boxes, or a 4–6 line snippet (patterns + findall / json.loads), proves understanding.

Tables for compare · marks to points

Regex vs JSON, JSON vs YAML, chunking strategies. 5 marks ≈ 5 points + diagram; 10 marks ≈ 8–10 points + diagram + example.